UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingenieria de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 01
Estructuras estáticas arreglo

Ciclo 2026-1

Agenda

1. Introducción
2. Definición
   - Características
   - Ventajas/Desventajas
3. Representación/Declaracion
   - Lógica
   - Física
4. Operaciones básicas
   - Análisis, diseño e implementación
   - Determinación de patrones de código para las solucione
5. Arreglos de caracteres
6. Ejemplos
7. Resumen
8. Referencias

1. Introducción

- **Arreglo.** Conjunto de elementos del mismo tipo conceptualmente relacionados.
- Se accede a los elementos a través del nombre y un índice: Los elementos están indexados/
- Es una estructura de datos básica estática.

2. Definición

- **Arreglo.** Es una estructura estática, contigua, homogénea, ordenada y finita de elementos.
  - **Estática.** Tiene un número máximo de elementos y debe ser declarado antes de definirse el arreglo
  - **Contigua.** Elementos uno a continuación de otro elemento.
  - **Homogénea.** Todos sus elementos son del mismo tipo.
  - **Ordenada.** Ocupan un determinado lugar en la estructura y podemos citar como primer, segundo, …, enésimo elemento.
  - **Finita.** Pues tiene un numero finito de elementos.

2. Características

- **Arreglo.** Tiene las siguientes características.
  - **Estática.** Tiene un número máximo de elementos y debe ser declarado antes de definirse el arreglo
  - **Tiempo acceso constante.** El tiempo de acceso al primer o ultimo elementos es el mismo. O(1)
  - **Acceso a los elementos.** A través de índice y de su nombre.
  - **Ordenada.** Ocupan un determinado lugar en la estructura y podemos citar como primer, segundo, …, enésimo elemento.
  - **Finita.** Pues tiene un numero finito de elementos.

2. Tipos

- **Arreglo.** Existe varios tipos de arreglos.
  - **Unidimensionales.** Denominados también vectores, se accesa a través de un nombre y un subíndice/índice.
  - **Bidimensionales.** Denominados también matrices, se accesa a través de su nombre y dos subíndices, el primero denominado fila y el segunda columna.
  - **Multidimensionales.** Se refiere a arreglos con más de dos dimensiones por ejemplo de subíndices i, j, k. En realidad, es un arreglo con múltiples vectores.

2. Ventajas y desventajas

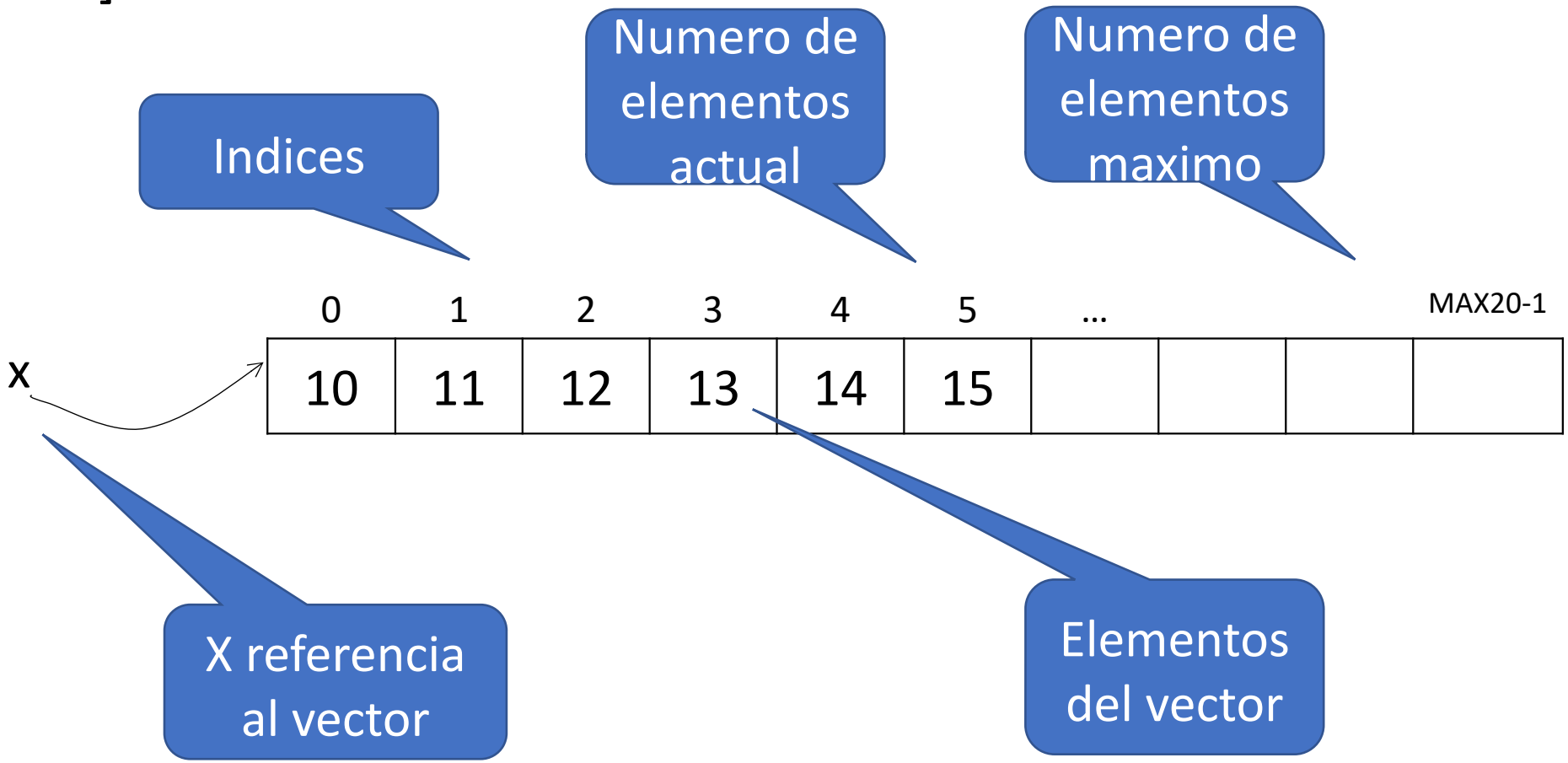
- **Ventajas.**
  - **Tiempo acceso constante.** El tiempo de acceso al primer o ultimo elementos es el mismo.
- **Desventajas.**
  - **Es volátil**
  - **Inserción/eliminación.** De elementos es costoso pues tiene que desplazar elementos a la derecha o izquierda respectivamente .
  - **Finita.** Pues tiene un numero finito de elementos.
  - Es un problema determinar el tamaño adecuado, si es pequeño no nos sirve y si muy grande es un desperdicio de memoria.

3. Declaracion y Representacion

- **Declaración.** Es muy simple, veamos:
  - **Dimensión del vector.**
    - MAX20 ← 20
    - MAXFILA ← 10, MAXCOL ← 10
  - **Tipo y nombre.**
    - tipoDato nombreVector[MAX]
    - tipoDato nombreMatriz[MAXFILA][MAXCOL]
- **Ejemplo.**
  - **Vector.** Crear un vector de enteros de 15 elementos.
    - MAX15 ← 15 #define MAX15 15
    - **ENTERO a[MAX15]** **int a[MAX15];**
  - **Matriz.** Crear una matriz de reales de 10 filas y 5 columnas
    - MAXFILA ← 10, MAXCOL ← 5 #define MAXFIL 1 #define MAXCOL 5
    - **REAL b[MAXFILA][MAXCOL]** **int [MAXCOL][MAXCOL];**

Arreglo unidimensional

- **Arreglo unidimensional.** Es un arreglo de una dimensión (vector) consta de un conjunto de celdas o datos bajo un mismo nombre x[MAX20].



Arreglo bidimensional

- **Arreglo Bidimensional.** Es un arreglo de dos dimensiones o matriz consta de un conjunto de celdas o datos bajo un mismo nombre x[MAXF][MAXC].

x

Indices columna

Numero maximo de columnas

X referencia a la matriz

Indices fila

Numero maximo de filas

	0	1	2	3	4	…			MAXC-1
0	00	01	02	03					
1	10	11	12	13					
2	20	21	22	23					
3	30	31	32	33					
4	40	41	42	43					
5									
…									
MAXF-1									

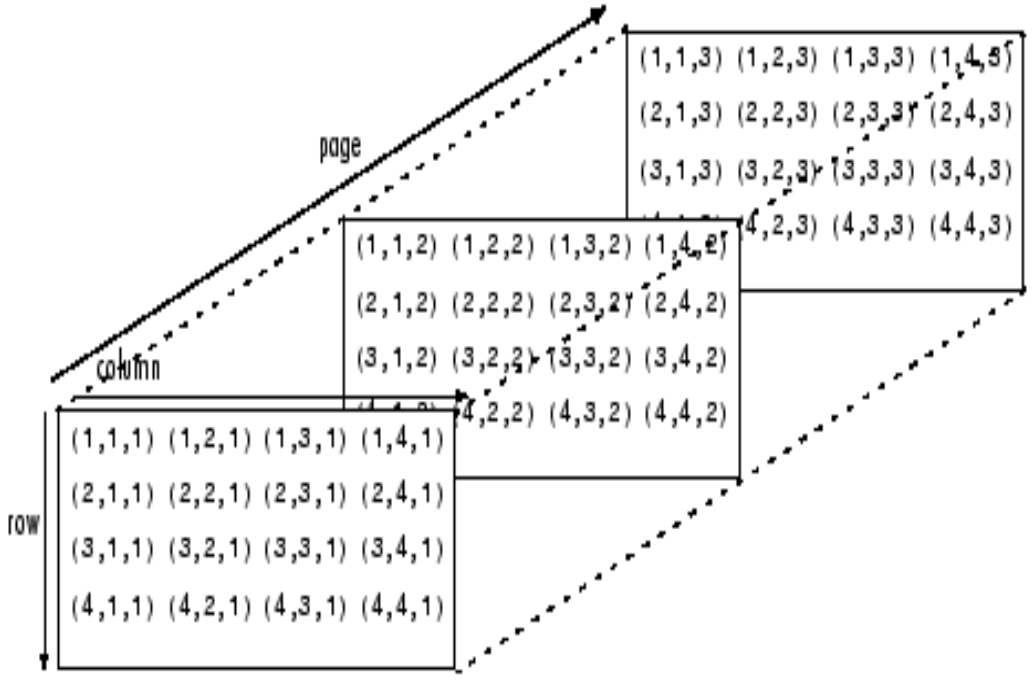
Elemento de la fila I, columna j

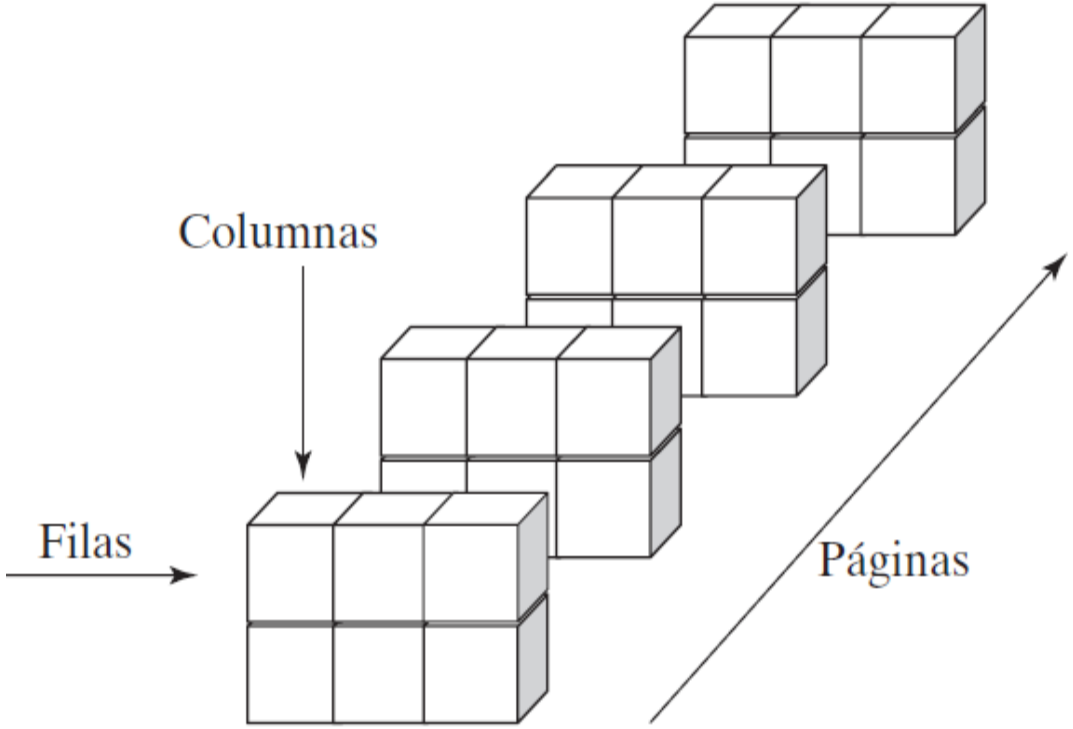
Numero actual de elemetos de la matriz (5 x 4)

Matriz de MAXF x MAXC

2. Arreglo multidimensional

- **Arreglo multidimensional.** Es un arreglo de más 2 dimensiones consta de un conjunto de celdas o datos bajo un mismo nombre. Ej. arreglo de 3 dimensiones: x[MAXF][MAXC][MAXP]

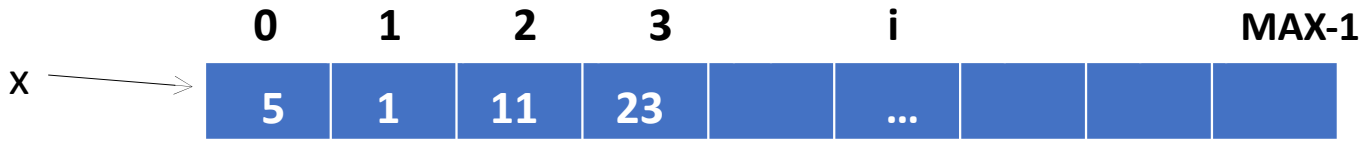




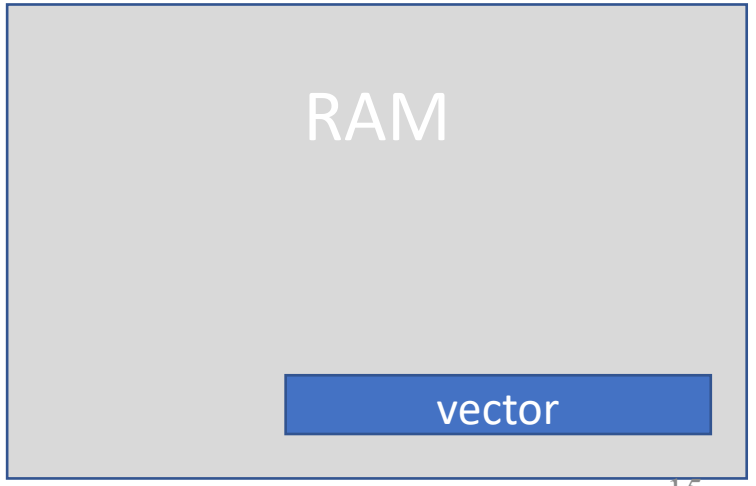
Fuente: MatLab

3. Representación

- **Representación.** Se refiere a la representación lógica y física o como se representa en memoria.
  - **Lógica.** Traduce el esquema de datos conceptual en un esquema de datos lógico para el arreglo. Facilita la estructuración de datos



  - **Física.** Es la representación del esquema lógico en un esquema físico donde subyace la estructura **(RAM)** .



4. Operaciones básicas

Operaciones Básicas. Son las operaciones elementales de acceso a los arreglos.

- **Actualización (**Ordenado y no Ordenado)
  - Inserción
  - Modificación
  - Borrado
- **Búsqueda**
  - Secuencial
  - Binaria
- **Métodos ordenamiento**
  - Cuadráticos
  - Logarítmicos

Clasificación de las operaciones. Para uso eficiente de la estructura

- **Constructoras**
  - crear Vector
- **Destructoras**
  - eliminarVector
- **Modificadoras**
  - insertarElemento
  - eliminarElemento
  - editarElemento
- **Analizadoras**
  - mostrarVector
  - buscarElemento
  - contarElementos
- **Persistencia**
  - salvarVector
  - recuperarVector

4. Implementación

- **Ejemplo.** Diseñe, especifique e implemente un vector de enteros y sus operaciones básicas.
- **Solución:** Análisis del problema en subproblemas, módulos u operaciones
  - Debemos accesar a la estructura para las diferentes operaciones.
  - Debemos pasar la información adecuada a cada una de esas operaciones.
  - Requiere de un ***diseño y especificación*** de la información que se les pasa.
  - Con el paso anterior podemos implementar dicha operación, modulo o subprograma.

Vector de enteros: operaciones básicas

- Crear
- Leer
- Mostrar
- adicionar
- Buscar
- Insertar
- Eliminar
- Editar
- …

4. Problemas

- **Ejercicio 1.** Se tiene un vector se pide implementar un subprograma para adicionar un dato al vector.
- **Ejercicio 2**. Se tiene un vector de calificaciones se pide determinar el promedio de calificaciones.
- .

5. Arreglo de caracteres

Caracteres:

- **Son los caracteres representados entre comillas simples.**
  - ‘0’, ‘1’, ‘2’, …, ‘8’, ‘9’
  - ‘a’, ‘b’, ‘c’, …, ‘y’, ‘z’ . ‘A’, ‘B’, ‘C’, …, ‘Y’, ‘Z’ .
  - ‘@’, ‘#’, ‘&’, …, ‘,’, ‘;’ . ‘.’, ….
- **Los caracteres son un tipo ordinal, o tipo enumerado usando el código ASCII** (American Standard Code for Information interchange)**.**

Cadenas.

- **Es un arreglo de caracteres:**
  - Puede incluir letras, dígitos, espacios o caracteres especiales.
  - Tiene un apuntador al primer elemento del arreglo.

3. Declaracion y Representacion

Declaración. Se declara como una variable de tipo CARACTER (char):

- **Dimensión del vector.**
  - STR20 ← 20
  - #define STR20 20
  - **CARÁCTER** color(STR20) ← “Anaranjado” // Al mas 19 caracteres (STR20 -1)
  - **char** color[STR20] = “anaranjado”;
  - **CARÁCTER** cad1[] ← “Hola mundo …”
  - **char** cad1[]/= “Hola mundo …” // Cadena inicializada
  - **CARÁCTER** *cad2 ← “Hola mundo cruel …”
  - **char** *cad2 = “Hola mundo cruel …”

3. Declaracion y Representacion

Lectura.

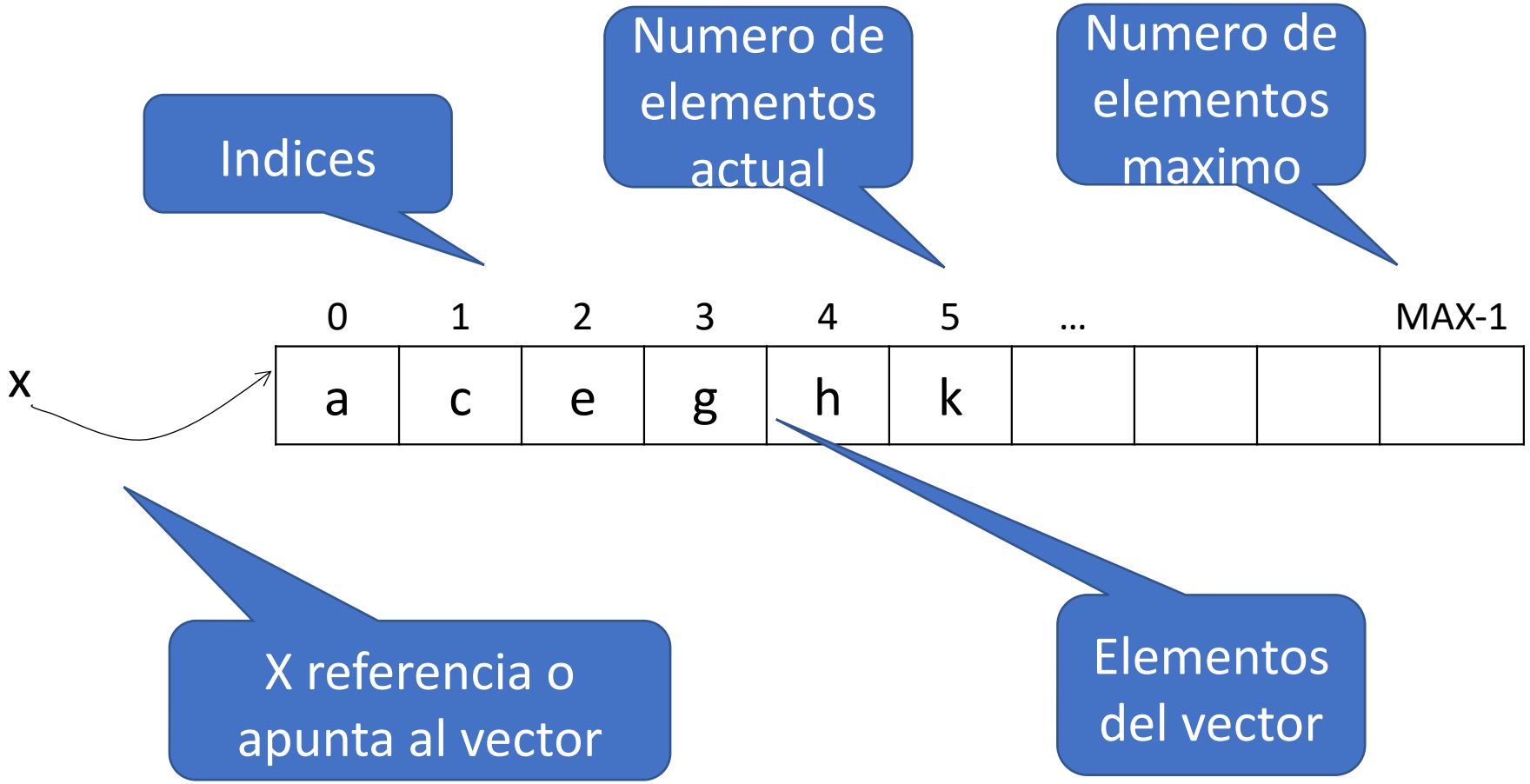
- **LEER (cad1)**
- scanf(“%s”, cad1)
- cin >> cad1;

Escritura

- **ESCRIBIR (cad1)**
- printf(“%s”, cad1)
- cout << cad1 <<endl;

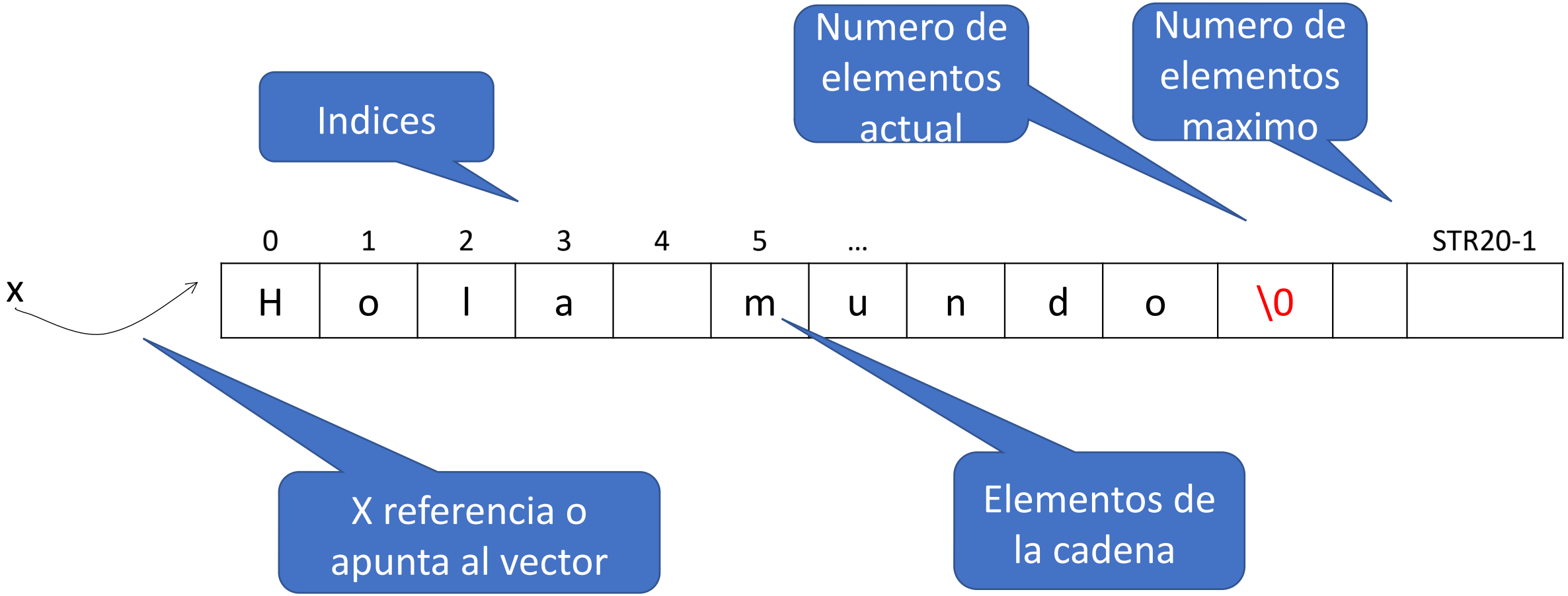
1. Introduccion

- **Arreglo caracteres.** Es un arreglo de una dimensión (vector) consta de un conjunto de celdas o datos bajo un mismo nombre x[MAX].



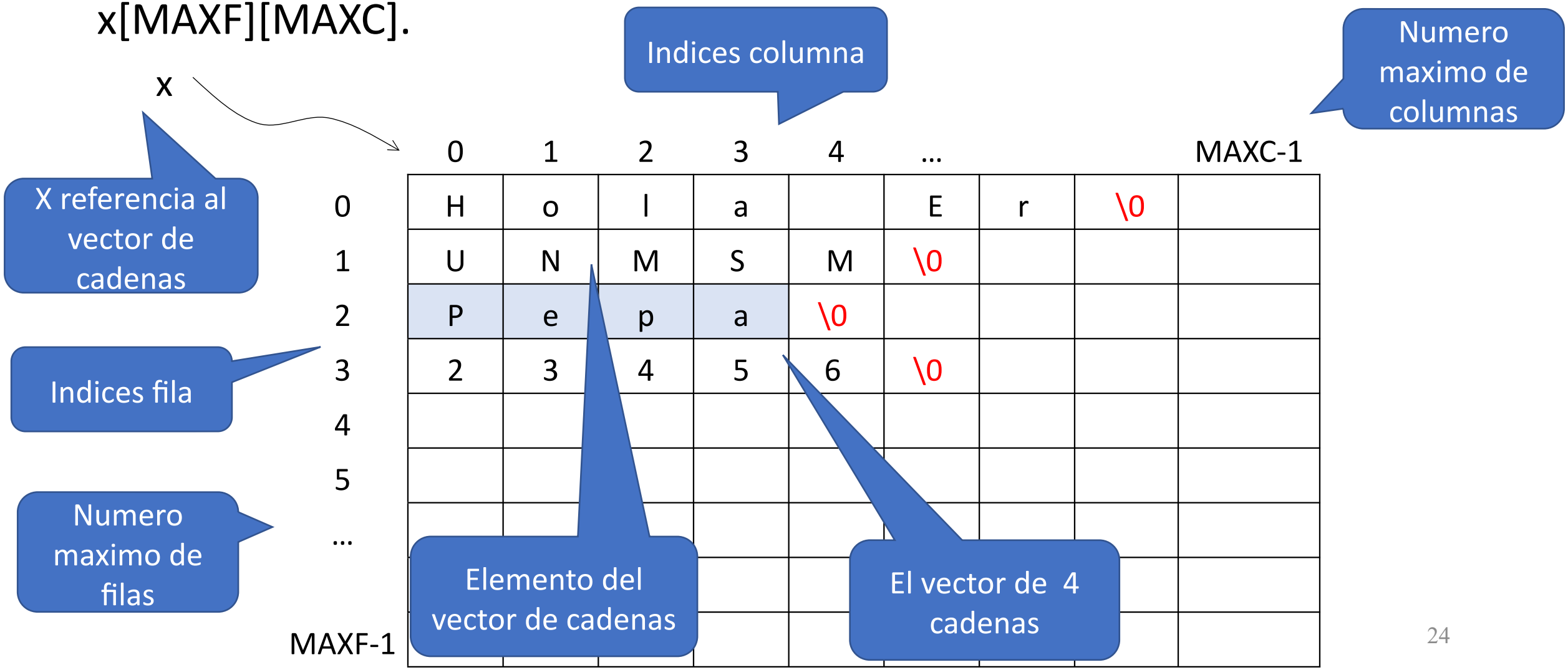
1. Introduccion

- **Cadena de caracteres.** Es un arreglo de una dimensión (vector) consta de un conjunto de celdas o datos bajo un mismo nombre x[MAX].



1. Introducción

- **Arreglo Bidimensional.** Es un arreglo de dos dimensiones o matriz consta de un conjunto de celdas o datos bajo un mismo nombre x[MAXF][MAXC].



1. Introducción

Tipo de datos estructura o compuesto.

- **Estructura Registro.** Es un tipo de dato compuesto definido por el usuario.
  - Está compuesta de elementos denominados “campos” de registro que pueden ser de diferente tipo de datos.
  - Permite crear nuevos “tipos de datos” lo cual facilita resolver problemas de la realidad.
    - PERSONA, PRODUCTO, ALUMNO, CURSO, etc

1. Introducción

Estructura de datos homogéneas y heterogéneas.

- **Arreglo.** Es una ED homogénea, todos sus elementos son del mismo tipo, por tanto, homogéneas.
- **Registros**. Es una ED heterogénea, sus elementos pueden ser de diferente tipo de dato.

2. Definición

Uniones.

Una variable de tipo unión es un conjunto de valores alterno que pueden almacenarse en una porción compartida de memoria (referencian a la misma dirección).

Las uniones se utilizan para ahorrar espacio en memoria..

Para almacenar los miembros de una unión se requiere de una zona igual a las más grande del conjunto de miembros de dicha estructura.

```
#define STR20  20
union DATOS{
      int codigo;
      char nombre[STR20];
      int edad;
      char sexo;
};
```

Declaración de variables

DATOS d1, d2;

Véase ejemplos

2. Definición

Enumeraciones.

Es similar a una estructura o una unión, cuyos miembros son constantes de valor entero, escritas como identificadores. Véase ejemplos

Formato:

enum NombreTipo {val1,val2, val3, …vanN};

Declaraciones de variables:

NombreTipo u1,u2,u3,…;

Ejemplo

enum DíasSemana {do, lu, ma, mi, ju, vi, sab};

num MesesAnio {ene,feb,mar,abr,may,jun,jul,sep,oct,nov,dic};

enum Calificacion{Mbueno,Bueno,regular, deficiente};

3. Registros

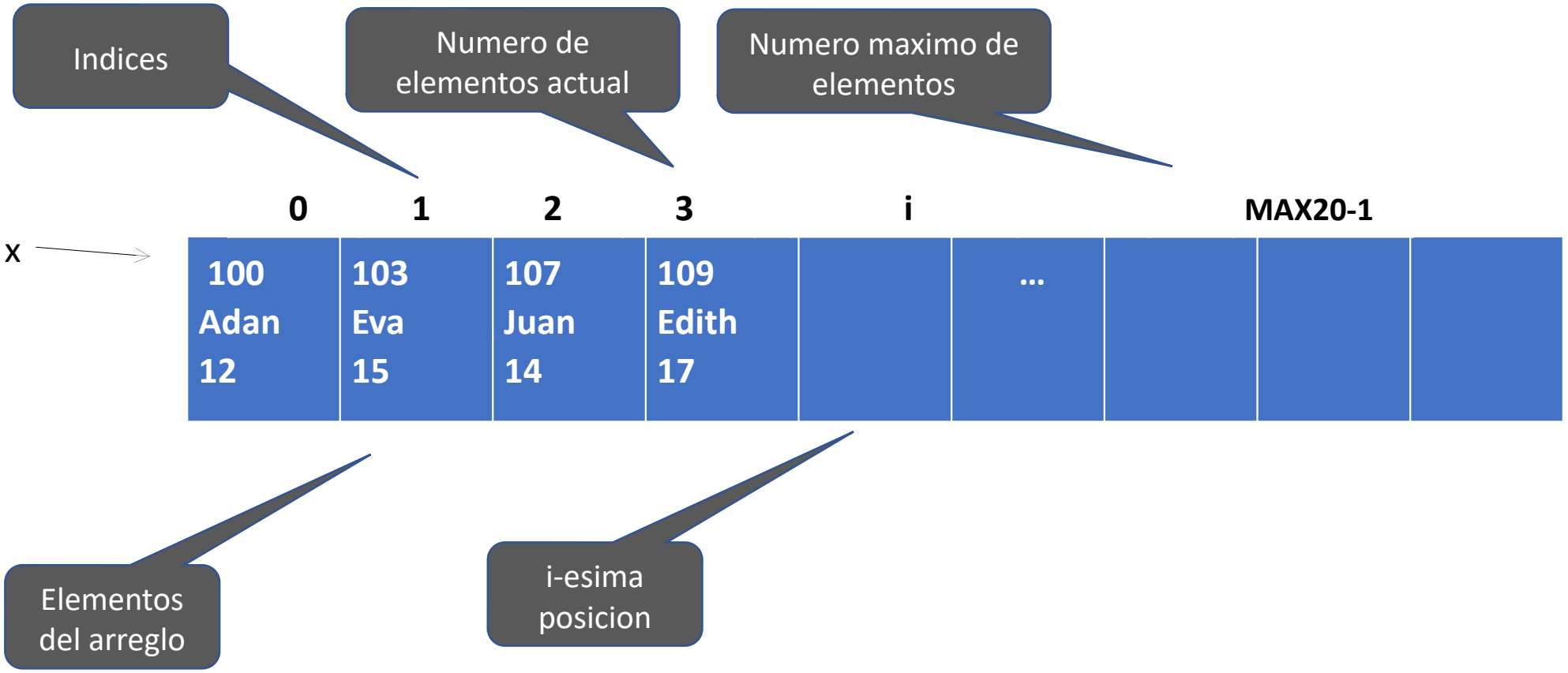
Estructura o tipo Registro.

Es una colección de variables predefinidas o simples (char, int, float, …) que pueden contener diferentes tipos de datos, denominado también Tipo de dato creado por el usuario. Llamado también Registros.

Se almacenan uno a continuación de otro.

3. Registros

- **Introducción.** Es un tipo de dato compuesto que consta de un conjunto de celdas o datos bajo un mismo nombre.



3. Registros

- **Registros**. Son tipo de datos estructurados, así como las **Uniones** y tipos **Enumerados**.
  - **Tipos abstractos.** Tipos de datos abstracto (TAD) o tablas como los vectores, matrices y de múltiples dimensiones. Son conjuntos de datos u objetos a los cuales se asocian operaciones.
  - **TAD.** Tipos abstractos de datos es un modelo que define valores y las operaciones que se pueden realizar sobre ellos.
    - Es una práctica que permite crear un tipo de dato abstracto (TAD) independiente de estructura y de lenguaje.
  - También se denomina, *Tipo de datos creados por el usuario*, pues permite crear tipos de datos de la realidad.
    - Alumno, Producto, TipoComplejo, TipoRacional, Persona, etc.

3. Registros: definición, declaración

- **Registro.** Es una estructura de datos que permite almacenar un conjunto de elementos no necesariamente del mismo tipo.
- Formato o sintaxis:

REGISTRO NombreRegistro

tipoDato1 nombreCampo1

tipoDato2 nombreCampo2

…

tipoDatoN nombreCampoN

FIN_REGISTRO

- En C los registros se definen o declaran de la siguien te manera
- Formato o sintaxis:

```
struct NombreRegistro{
    tipoDato1   nombreCampo1;
    tipoDato2   nombreCampo2;
    …
    tipoDatoN   nombreCampoN;
};
```

3. Definición, declaración de tipos registro

Registro. Declaración de variables.

- Formato o sintaxis:

MAX ← 50

REGISTRO NombreRegistro

tipoDato1 nombreCampo1

tipoDato2 nombreCampo2

…

tipoDatoN nombreCampoN

FIN_REGISTRO

Declaración de variables:

NombreRegistro nv1, nv2, nv3[MAX]

- En C los registros se definen o declaran de la siguien te manera
- Formato o sintaxis:

```
#define MAX 50
struct NombreRegistro{
    tipoDato1    nombreCampo1;
    tipoDato2    nombreCampo2;
    …
    tipoDatoN    nombreCampoN;
};
```

Declaracion de variables:

NombreRegistro nv1, nv2, nv3[MAX];

3. Definición, acceso a los campos

- **Registro.** Acceso a los campos se a través del operador punto.
- Formato o sintaxis:

MAX50 ← 50

REGISTRO NombreRegistro

tipoDato1 nombreCampo1

tipoDato2 nombreCampo2

…

tipoDatoN nombreCampoN

FIN_REGISTRO

NombreRegistro nv1, nv2, nv3[MAX50]

nvi.nombreCampo

- En C los registros se definen o declaran de la siguien te manera
- Formato o sintaxis:

```
#define MAX50 50
struct NombreRegistro{
    tipoDato1   nombreCampo1;
    tipoDato2   nombreCampo2;
    …
    tipoDatoN   nombreCampoN;
};
NombreRegistro nv1, nv2, nv3[MAX50];
nvi.nombreCampo;
```

3. Registro, ejemplo

- **Registro.** Por ejemplo CLIENTE
- Formato o sintaxis:

STR20 ← 20

STR80 ← 80

REGISTRO CLIENTE

ENTERO	dni
CARÁCTER	ruc[STR20]
ENTERO	noCuenta
CARACTER	nombre[STR80]
CARÁCTER	domicilio[STR80]
REAL	saldo

FIN_REGISTRO

- En C los registros se definen o declaran de la siguien te manera
- Formato o sintaxis:

```
#define STR20 20
#define STR80 80
struct CLIENTE {
    long int    dni;
    Char        ruc[STR20];
    long int    noCuenta;
    char        nombre[STR80];
    char        domicilio[STR80];
    float       saldo;
};
```

3. Registros, ejemplo

- **Ejemplo.** Crear la estructurq alumno

MAX50 ← 50

STR20 ← 20

REGISTRO ALUMNO

ENTERO	cod
CARACTER	nombre[STR20]
REAL	nota

FIN_REGISTRO

```
ACCION principal()
    ALUMNO  a, b, c[MAX50]
    //Rutina de lectura
    LEER(a.cod)
    LEER(a.nom)
    LEER(a.nota)
    //Rutina de escritura
    ESCRIBIR(a.cod)
    ESCRIBIR(a.nom)
    ESCRIBIR(a.nota)
FIN_ACCION
```

3. Registro, ejemplo

- **Ejemplo.** Crear la estructurq alumno

```
#include <…>
#define MAX50 50
#define STR20  20
Using namespace std;
struct  ALUMNO{
      int        cod
      char       nombre[STR20]
      float      nota
};
```

```
int main()
{
      ALUMNO  a, b, c[MAX50];
      //Rutina de lectura
      scanf("%d", a.cod);
      scanf("%s", a.nom);
      scanf("%f", a.nota);
      //Rutina de escritura
      printf("%d", a.cod);
      printf("%s", a.nom);
      printf("%f", a.nota);
      system("pause");
      return(0);
}
```

4. Combinaciones con vectores

- **Registros.** Siendo un nuevo tipo de dato tiene las mismas prerrogativas que un tipo de dato predefinido del lenguaje:
  - **Vector de registros.** Se puede crear vectores de registros.
  - **Registros con vectores.** Pue contener como campos del registro tipos datos estructurados como vectores.
  - **Registros anidados.** Puede contener con campos del registro tipos de dato de tipo registro.

4. Registros, combinaciones

- **Ejemplo.** Crear la estructurq alumno

MAX50 ← 50

STR20 ← 20

REGISTRO ALUMNO

ENTERO	cod
CARACTER	nombre[STR20]
REAL	nota

FIN_REGISTRO

ACCION principal()

ALUMNO a, b, c[MAX50]

ENTERO nc

crearVector(c, na)

leerVector(c, na)

mostrarVector(c, na)

FIN_ACCION

4. Registros, combinaciones

- **Ejemplo.** Crear la estructurq alumno con vector practicas

MAX4 ← 4

MAX50 ← 50

STR20 ← 20

REGISTRO ALUMNO

ENTERO	cod
CARACTER	nombre[STR20]
REAL	**pract[MAX4]**

FIN_REGISTRO

ACCION principal()

ALUMNO a, b, c[MAX50]

ENTERO nc

crearVector(c, na)

leerVector(c, na)

mostrarVector(c, na)

FIN_ACCION

4. Registros, combinaciones

- **Ejemplo.** Crear la estructurq alumno con con la estructura fecha

MAX4 ← 4

MAX50 ← 50

STR20 ← 20

REGISTRO FECHA

ENTERO dd

ENTERO mm

ENTERO aaaa

FIN_REGISTRO

REGISTRO ALUMNO

ENTERO cod

CARACTER nombre[STR20]

REAL pract[MAX4]

FECHA fnac

FIN_REGISTRO

ACCION principal()

ALUMNO a, b, c[MAX50]

ENTERO nc

crearVector(c, na)

leerVector(c, na)

mostrarVector(c, na)

FIN_ACCION

5. Operaciones básicas

- **Operaciones.** Son las operaciones elementales de acceso a los arreglos, lo cual nos permitirá utilizar la estructura eficientemente y a resolver problemas.
  - **Constructoras**
    - crear Vector
  - **Destructoras**
    - eliminarVector
  - **Modificadoras**
    - insertarElemento
    - eliminarElemento
    - editarElemento
  - **Analizadoras**
    - mostrarVector
    - buscarElemento
    - contarElementos
  - **Persistencia**
    - salvarVector
    - recuperarVector

6. Ejercicios, Aplicaciones

- **Ejercicio 1.** Analice, diseñe, especifique e implemente las operaciones básicas de vector de registros de alumnos cuyos campos son código, nombre notas de practica, examen parcial y final.
- **Ejercicio 2.** Crear las operaciones básicas para un conjunto de de enteros.
- **Ejercicio 3.** Crear las operaciones básicas para un umero racional (fracciones)

RETO:

- Ejercicio 4. Crear una aplicación de un sistema de matricula.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesaro en Lima, 19.05.2020, 1820 horas], http://c.conclase.net/ficheros/
- Diseno.[Ultima actualización 07.11.2015] [Accesado en Lima, 18.03.2020, 1820 horas], https://es.wikipedia.org/wiki/Dise%C3%B1o
- Ingenieria.[Ultima actualización 24.02.2014] [Accesado en Lima, 18.03.2020, 1910 horas], https://es.wikipedia.org/wiki/Ingenier%C3%ADa